Counting Sort x Bucket Sort

Ludmila Lima do Espirito Santo e Matheus Silva Pains

Projeto e Análise de Algoritmos 1 (CMP 1065)
Professor: Prof. MSc. Anibal Santos Jukemura
Experimentos: 22 de outubro de 2026.

Este relatório apresenta a implementação em C, a análise assintótica e a comparação experimental dos algoritmos. Os itens seguem a ordem A a G da atividade.

A. Melhor e pior caso em notação Θ

Adotam-se n elementos inteiros sem sinal de 32 bits, faixa conhecida de K chaves em [0, K), com K = 10n, e b = n baldes. O número de chaves possíveis K é mantido para cada tamanho, inclusive quando a entrada usa somente 0 e 1. Esses parâmetros são parte da análise, e não valores recalculados a partir do máximo observado.

Counting Sort: contagens acumuladas

A implementação inicializa K contadores, conta n ocorrências, acumula as K contagens, percorre a entrada de trás para frente para construir a saída e copia os n elementos. Portanto, melhor e pior caso são Θ(n + K). Com K = 10n, ambos são Θ(n). A ordem de entrada e a concentração dos valores não eliminam a passagem por toda a faixa. A memória auxiliar é Θ(n + K). A passagem reversa preserva a ordem relativa de chaves iguais, caracterizando a versão estável. [1]

Bucket Sort: Insertion Sort dentro de cada balde

O índice é floor(valor × b / K); com os parâmetros adotados, equivale a valor / 10. Os baldes são segmentos de um vetor auxiliar, dimensionados por contagens e posições acumuladas, sem reservar n posições para cada balde. Inicialização, distribuição estável e reunião custam Θ(n + b). A memória auxiliar é Θ(n + b).

Melhor caso: Θ(n + b), ou Θ(n) com b = n. Cada balde recebe um elemento. O Insertion Sort não desloca elementos e as demais etapas são lineares. Esse é um exemplo suficiente de melhor caso; baldes já ordenados também podem produzir custo linear.

Pior caso: Θ(n² + b), ou Θ(n²) com b = n. A entrada contém primeiro n/2 valores 1 e depois n/2 valores 0. Todos pertencem ao primeiro balde. Cada zero desloca os n/2 valores 1, totalizando $n^2/4$ deslocamentos para os tamanhos pares utilizados. Esse custo atinge a mesma ordem quadrática do limite superior do Insertion Sort.

Sob distribuição aproximadamente uniforme e b proporcional a n, o custo esperado do Bucket Sort com Insertion Sort é linear. Essa afirmação depende da distribuição e não substitui a análise do pior caso. A escolha do algoritmo interno é explícita: o exemplo do Hello Algo usa qsort; este trabalho adota Insertion Sort para estudar a degeneração quadrática. [2]

Implementação e protocolo

B. Programa em linguagem C

Os algoritmos recebem cópias do mesmo vetor de uint32_t. Counting Sort usa contagens acumuladas e saída auxiliar; Bucket Sort usa distribuição por intervalos, preserva a ordem de chegada e aplica Insertion Sort em cada segmento. A comparação utiliza o mesmo domínio de entrada e o mesmo tipo numérico.

Estrutura das operações

Counting Sort: zerar K contadores; contar as chaves; acumular as contagens; percorrer a entrada em sentido reverso, posicionando cada valor na saída; copiar a saída e liberar a memória.

Bucket Sort: contar ocupações; calcular os limites dos segmentos; distribuir preservando a ordem; ordenar cada segmento com Insertion Sort; copiar o resultado e liberar a memória.

Medição e validação

O relógio QueryPerformanceCounter mede a chamada completa de cada ordenação, incluindo alocação, processamento, cópia de saída e liberação de memória. Geração, cópia das entradas, ordenação de referência com qsort, conferência e gravação no CSV ficam fora do intervalo cronometrado.

Cada saída é conferida quanto à ordem crescente e à igualdade exata com o resultado de qsort. Assim, não basta produzir um vetor crescente: os valores e suas multiplicidades devem coincidir com a entrada. Todas as 162 medições registradas passaram nas duas verificações.

Entradas e repetições

São nove tamanhos: 10, 100, 1.000, 5.000, 10.000, 50.000, 100.000, 500.000 e 1.000.000. Para cada tamanho e cenário realizam-se três repetições por algoritmo. A ordem de execução dos algoritmos é alternada entre as repetições, e as medições são sequenciais. O CSV registra as sementes e o tempo individual de cada execução.

Aleatório: valores produzidos por SplitMix64 e reduzidos módulo K. Isso fornece uma distribuição pseudoaleatória aproximadamente uniforme; a redução modular pode introduzir viés discreto desprezível neste domínio, e não representa todos os padrões de dados possíveis.

Melhor para Bucket Sort: chaves $10i + 5$, para i de 0 a $n - 1$, embaralhadas. Cada balde recebe exatamente um elemento. **Pior para Bucket Sort:** metade de valores 1 seguida de metade de valores 0; todos ficam no primeiro balde.

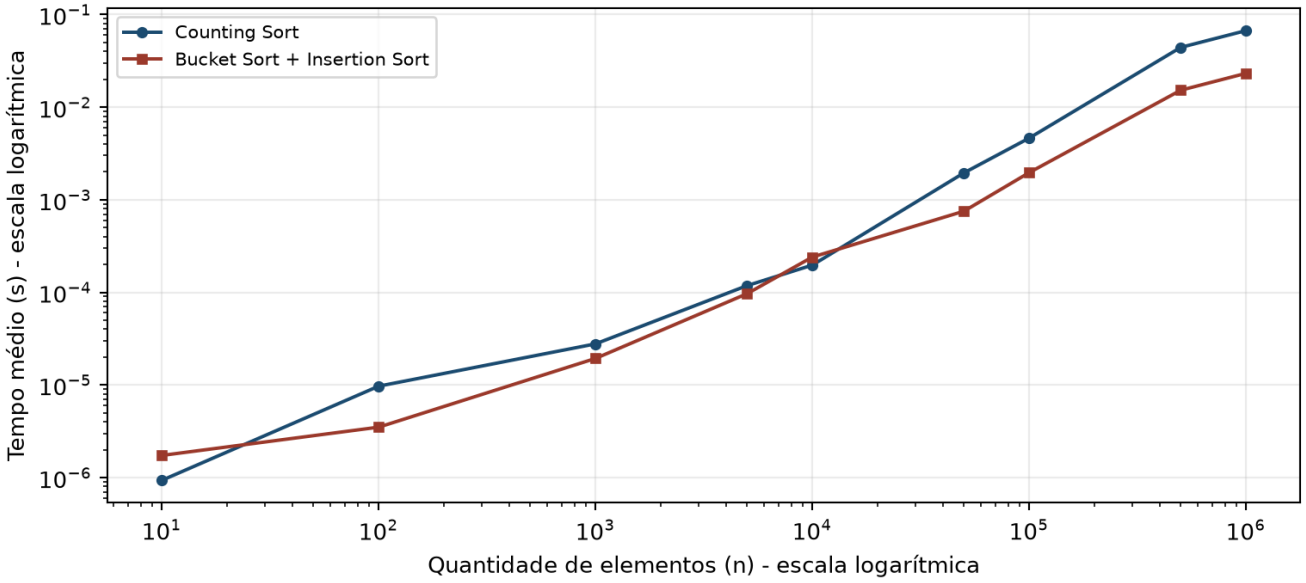
Os nomes “melhor” e “pior” designam cenários construídos para Bucket Sort. Para Counting Sort, com a faixa K fixada, todos eles conservam $\Theta(n + K)$; os tempos medidos ainda podem variar por acesso à memória, cache e custos do sistema.

As tabelas seguintes apresentam médias aritméticas de três execuções em segundos. Os tempos individuais e as medidas de dispersão estão na planilha e no CSV. Não se somam tempos de execuções distintas. Medidas muito pequenas são mais sensíveis à resolução e ao ruído do ambiente.

C. Entradas aleatórias

Valores pseudoaleatórios no domínio comum [0, 10n). Os dois algoritmos recebem exatamente os mesmos valores em cada repetição.

Elementos (n)	Counting Sort (s)	Bucket Sort (s)
10	0,000000933	0,000001733
100	0,000009700	0,000003500
1.000	0,000027767	0,000019333
5.000	0,000118200	0,000096733
10.000	0,000196833	0,000240433
50.000	0,001948967	0,000751933
100.000	0,004632667	0,001963267
500.000	0,044200733	0,015235300
1.000.000	0,067102500	0,023190467

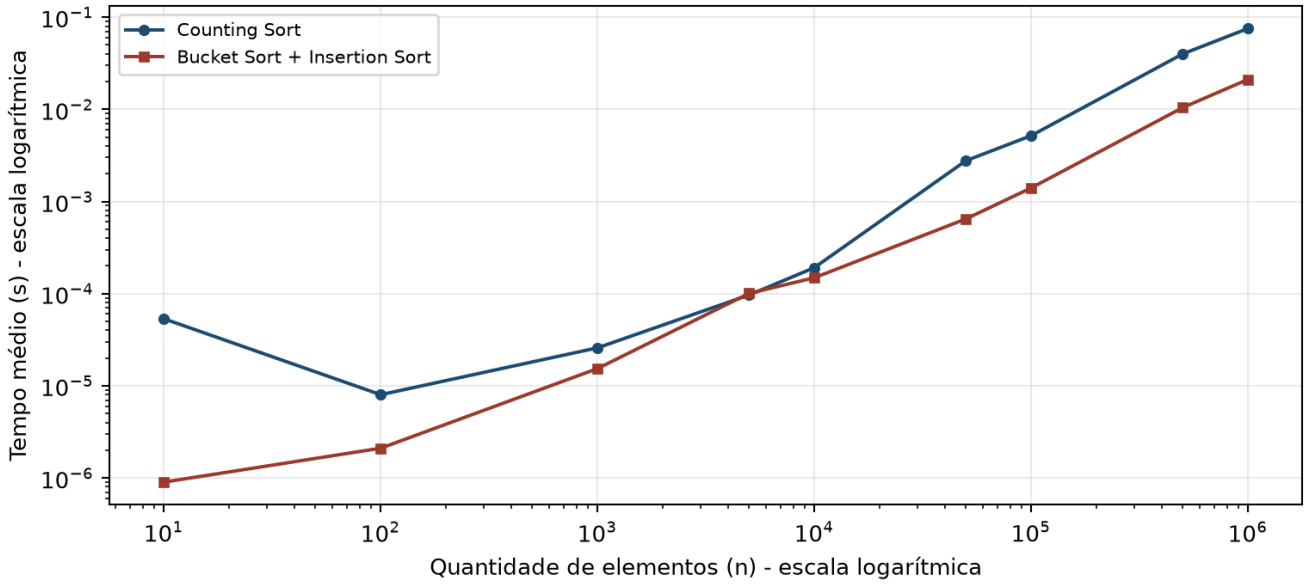


Fonte: medições reais em db-resultados/comparacao_medicoes.csv; média de três repetições por ponto. Escalas logarítmicas nos dois eixos, para permitir a leitura conjunta de tamanhos e tempos com ordens de grandeza diferentes.

D. Cenário de melhor caso

Um elemento por balde, com entrada embaralhada. Counting Sort percorre a faixa de 10n chaves e Bucket Sort não realiza deslocamentos internos.

Elementos (n)	Counting Sort (s)	Bucket Sort (s)
10	0,000053367	0,000000900
100	0,000008000	0,000002100
1.000	0,000025800	0,000015367
5.000	0,000096600	0,000100600
10.000	0,000191267	0,000148633
50.000	0,002769067	0,000646700
100.000	0,005179500	0,001402533
500.000	0,039917600	0,010436367
1.000.000	0,075403800	0,021125400



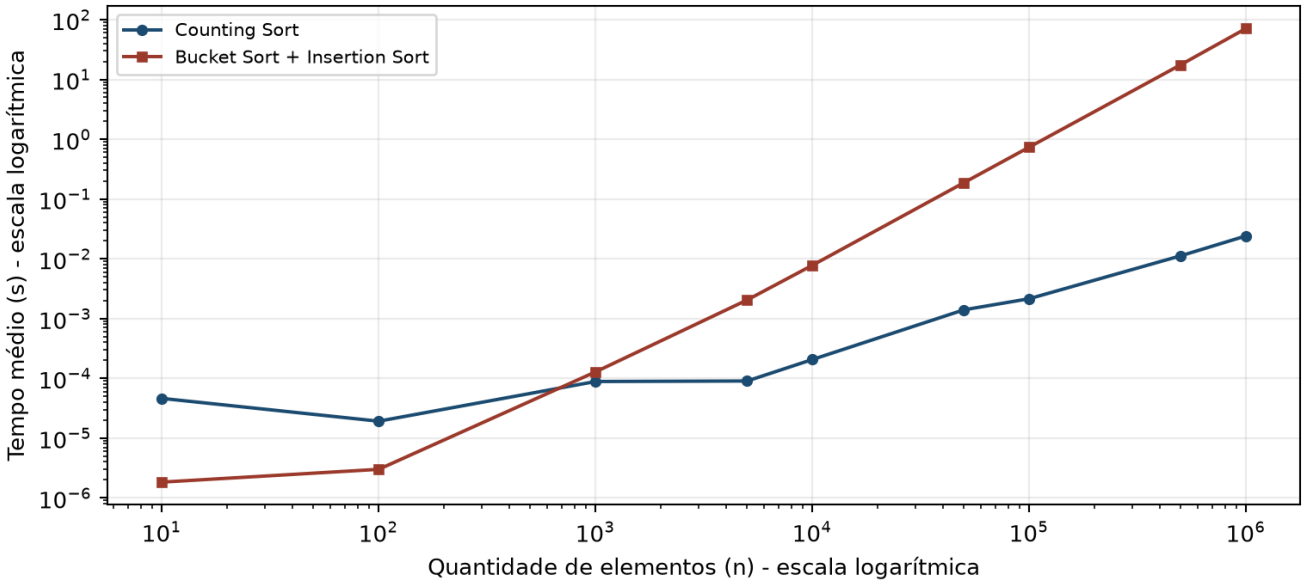
Fonte: medições reais em db-resultados/comparacao_medicoes.csv; média de três repetições por ponto. Escalas logarítmicas nos dois eixos, para permitir a leitura conjunta de tamanhos e tempos com ordens de grandeza diferentes.

O tamanho 1.000.000 complementa os oito tamanhos obrigatórios deste cenário, atendendo à possibilidade prevista no item F.

D. Cenário de pior caso

Todos os valores pertencem ao primeiro balde, com todos os 1 antes dos 0. No Bucket Sort, o Insertion Sort executa $n^2/4$ deslocamentos; Counting Sort mantém sua estrutura linear em n + K.

Elementos (n)	Counting Sort (s)	Bucket Sort (s)
10	0,000046100	0,000001833
100	0,000019167	0,000003000
1.000	0,000088367	0,000127933
5.000	0,000090033	0,002039667
10.000	0,000205900	0,007764167
50.000	0,001396767	0,186937867
100.000	0,002141800	0,734888200
500.000	0,011192100	17,560466000
1.000.000	0,024073767	71,014851767



Fonte: medições reais em db-resultados/comparacao_medicoes.csv; média de três repetições por ponto. Escalas logarítmicas nos dois eixos, para permitir a leitura conjunta de tamanhos e tempos com ordens de grandeza diferentes.

O tamanho 1.000.000 complementa os oito tamanhos obrigatórios deste cenário, atendendo à possibilidade prevista no item F.

Máquina e interpretação

E. Computador utilizado

CPU: Intel Core i5-13400F, 13ª geração, 10 núcleos físicos e 16 processadores lógicos. RAM instalada: 16 GiB, um módulo, velocidade informada de 2.666 MHz. Fabricante do computador: ASUS; modelo informado pelo firmware: “System Product Name”. Placa-mãe: ASUSTeK PRIME H610M-E D4. Sistema: Windows 11 Pro, 64 bits, versão 10.0.26100.

Dados coletados via PowerShell/CIM e preservados em doc-referencia/pc-utilizado.txt. A frequência de 2.500 MHz informada por Win32_Processor não representa o limite de turbo. Compilador: GCC 15.2.0 (MSYS2), linguagem C11, otimização -O2.

F. Testes adicionais

Foram executados também os cenários de melhor e pior caso com 1.000.000 de elementos, além do teste aleatório obrigatório nesse tamanho. O conjunto final contém 2 algoritmos × 3 cenários × 9 tamanhos × 3 repetições = 162 medições.

G. Conclusões dos experimentos

Counting Sort apresenta $\Theta(n + K)$ tanto no melhor quanto no pior caso da versão implementada. Com $K = 10n$, o crescimento assintótico é linear, embora os tempos variem entre entradas por efeitos de acesso à memória.

Bucket Sort depende da ocupação dos baldes e da ordenação interna. Um elemento por balde evita deslocamentos; a concentração de 1 e 0 em um balde produz $n^2/4$ deslocamentos e pior caso $\Theta(n^2)$ com Insertion Sort.

Para 1.000.000 de entradas aleatórias, as médias foram 0,067102500 s para Counting Sort e 0,023190467 s para Bucket Sort. Nesse ponto, Bucket Sort + Insertion Sort apresentou o menor tempo médio; o resultado vale para esta implementação, máquina e domínio.

No cenário concentrado com 1.000.000 de elementos, Counting Sort levou em média 0,024073767 s e Bucket Sort 71,014851767 s. A diferença evidencia a importância da distribuição e do custo da ordenação interna.

Discussão e referências

Limitações e questões para discussão

Os algoritmos receberam cópias idênticas das entradas e todas as 162 execuções tiveram saída ordenada e igual à referência qsort. A correção foi verificada fora do tempo medido.

O experimento fixa a faixa em dez vezes o tamanho da entrada. Não demonstra vantagem universal: uma faixa muito maior penaliza tempo e memória do Counting Sort; outra ordenação interna ou outro mapeamento pode alterar o comportamento do Bucket Sort.

Três repetições fornecem uma descrição inicial da variação; não sustentam uma caracterização estatística ampla. Não foram controladas afinidade de CPU, frequência, cache, serviços do sistema ou paginação. A ordem alternada reduz um viés simples de ordem, mas não elimina interferências. A taxa de crescimento teórica não é comprovada apenas por uma curva de tempos.

O uso de uint32_t permite comparar os algoritmos no mesmo domínio. Os resultados históricos de Bucket Sort com double foram preservados separadamente e não foram combinados com as novas médias, gráficos e conclusões comparativas. Os custos de memória são analisados teoricamente; não foi medido o pico real de RAM.

Questões para a discussão em sala: como muda o resultado se K crescer muito além de n? O que ocorre ao reduzir a quantidade de baldes? E se os dados se concentrarem em poucos intervalos? Qual seria o efeito de substituir Insertion Sort por uma ordenação com pior caso O(n log n)?

Referências bibliográficas

[1] MORIN, Pat. **Open Data Structures (in C++)**. Edição 0.1Gβ. Capítulo 11, subseção 11.2.1, “Counting Sort”, p. 239-241; posições 253-255 do PDF. Disponível em: https://opendatastructures.org/ods-cpp.pdf. Acesso em: 22 out. 2026.

[2] KRAHETS e colaboradores. **Hello Algo**. Versão 1.3.0, edição em inglês com exemplos em C. Seção 11.8, “Bucket Sort”, p. 266-269; posições 273-276 do PDF. Discussão do pior caso: seção 11.11, p. 279, posição 286 do PDF. Disponível em: https://www.hello-algo.com/en/chapter_sorting/bucket_sort/. Acesso em: 22 out. 2026.

Arquivos que acompanham a entrega

cod-implementacao/: fontes C, validação e roteiro de execução.
db-resultados/comparacao_medicoes.csv: 162 medições individuais.
db-resultados/comparacao_resultados.xlsx: planilha, resumos e gráficos.
doc-referencia/: livros, enunciado e configuração do computador.
doc-entrega/conclusoes.pdf: material para impressão e distribuição.
web-apresentacao/: apresentação para dispositivos móveis.

Apresentação: https://greenbalding.github.io/CountingSort_BucketSort/